

Appunti di Basi di Dati

Emanuele Romano

Indice

1	Progettazione Concettuale	1
1.1	Progettazione Concettuale con Class Diagrams	1
1.1.1	Ruoli nelle Associazioni	2
1.1.2	Navigabilità e Direzionalità	2
1.1.3	Tipi Enumerativi (Enumerazioni)	3
1.1.4	Classi di Associazione	3
1.1.5	Grado di un'Associazione e Relazioni Ternarie	5
1.1.6	Aggregazione e Composizione	7
1.1.7	Generalizzazioni/Specializzazioni	8
1.2	La Documentazione di Supporto: I Dizionari dei Dati	9
2	Modello Relazionale e Progettazione Logica	11
2.1	Introduzione	11
2.2	Concetti Fondamentali	11
2.3	Le Chiavi	13
2.4	Traduzione delle Associazioni	14
2.4.1	Associazioni 1 a 1	14
2.4.2	Associazioni 1 a Molti (1:N)	16
2.4.3	Associazioni Molti a Molti (N:N)	17
2.5	Ristrutturazione dello Schema Concettuale	18
2.5.1	Analisi delle Chiavi: Chiavi Naturali vs Surrogate	19
2.5.2	Analisi degli Attributi Derivati e delle Ridondanze	20
2.5.3	Analisi delle Associazioni Uno a Uno: Fusione vs Separazione	21
2.5.4	Analisi degli Attributi Strutturati	22
2.5.5	Analisi degli Attributi a Valore Multiplo	24
2.6	Codifica delle Gerarchie (Ereditarietà)	25
2.7	Riepilogo: Il Ciclo di Vita della Progettazione dei Dati	26
3	Normalizzazione Relazionale	28
3.1	Ridondanze e Anomalie	28
3.2	Dipendenze Funzionali	29
3.3	La Forma Normale di Boyce-Codd (BCNF)	30
3.3.1	Decomposizioni	31
3.3.2	Qualità delle Decomposizioni	32
3.4	La Terza Forma Normale (3NF)	33
3.5	Le Forme Normali 1NF e 2NF	34
4	Introduzione a SQL e Definizione dei Metadati	36
4.1	Nozioni introduttive	36
4.2	I Domini di Base (Tipi di Dato) nello Standard SQL	37
4.3	Operatori Logici e di Confronto	39
4.4	Istruzioni DDL	40
4.4.1	Creazione di uno schema logico	40

4.4.2	Creazione di una tabella	41
4.4.3	I Vincoli Intra-attributo (o in-line)	41
4.4.4	I Vincoli di Tabella (Out-of-line)	42
4.4.5	Classificazione Concettuale dei Vincoli	44
4.4.6	Integrità Referenziale e Chiavi Esterne (FOREIGN KEY)	46
4.4.7	Identificatori Univoci: Chiavi Surrogate e il costrutto IDENTITY	48
5	SQL e Algebra Relazionale	51
5.1	Proiezioni	52
5.1.1	La Proiezione in SQL: Il comando SELECT	53
5.2	Selezioni	53
5.2.1	La Selezione in SQL: La clausola WHERE	54
5.2.2	Attributi Parziali e Logica Ternaria	55
5.3	Composizione di Proiezioni e Selezioni	56
5.3.1	La Composizione in SQL e l'Ordine Logico di Esecuzione	57
5.4	Qualificazione degli Attributi	58
5.5	Ridenominazione	58
5.5.1	La Ridenominazione in SQL: L'operatore AS	58
5.6	Operazioni Insiemistiche	59
5.6.1	Le Operazioni Insiemistiche in SQL	61
5.7	Il Prodotto Cartesiano e le Giunzioni (Join)	62
5.7.1	I Join in SQL	64
5.7.2	Il Prodotto Cartesiano in SQL: CROSS JOIN	65
5.8	Giunzioni Esterne (Outer Join)	66
5.8.1	Le Giunzioni Esterne in SQL	66
5.8.2	Il pattern "Anti-Join": Isolare le assenze	67
5.9	Raggruppamento e Funzioni di Aggregazione	67
5.9.1	Le Funzioni di Aggregazione in SQL: GROUP BY	69
5.9.2	Filtrare i Gruppi: La clausola HAVING	70
5.10	L'Ordine Logico di Esecuzione in SQL	71
5.11	Le Viste (Views): Tabelle Virtuali	72
5.12	Interrogazioni Nidificate (Subqueries)	73
5.12.1	Subqueries in WHERE (filtri dinamici)	73
5.12.2	Subqueries in FROM (Tabelle Derivate o Inline Views)	76
5.12.3	Subqueries in SELECT (Colonne Calcolate)	77
5.13	Viste vs Interrogazioni Nidificate	78
5.14	Ordinamento dei Risultati: La clausola ORDER BY	79
5.15	Espressioni e Funzioni Predefinite nella SELECT	80
5.15.1	Uso di Espressioni Costanti	81
5.15.2	Panoramica delle Funzioni Predefinite Standard	81
5.16	Manipolazione dei Dati (DML): INSERT, UPDATE, DELETE	82
5.16.1	L'istruzione INSERT	82
5.16.2	L'istruzione UPDATE	83
5.16.3	L'istruzione DELETE	83

6	SQL Avanzato in PostgreSQL	85
6.1	Controllo del Result Set: Limit, Offset e Nulls	85
6.1.1	Taglio e Paginazione: <code>LIMIT</code> e <code>OFFSET</code>	85
6.1.2	Gestione delle Incognite nell'Ordinamento: <code>NULLS FIRST / LAST</code>	86
6.2	DML Avanzato in PostgreSQL: <code>RETURNING</code> e <code>UPSERT</code>	86
6.3	Gestione Dati Semi-Strutturati: Il tipo <code>JSONB</code>	87
6.4	Common Table Expressions (CTE): Modularità e Ricorsione	88
6.4.1	La clausola <code>WITH</code> (Sintassi e semantica)	88
6.4.2	CTE Multiple e Concatenate (Data Pipelining)	89
6.4.3	Cenni alle CTE Ricorsive (<code>WITH RECURSIVE</code>)	90
6.5	Logica Condizionale e Gestione delle Incognite	92
6.5.1	Il costrutto <code>CASE WHEN</code> (Semplice vs Ricercato)	93
6.5.2	Pivoting dei Dati (Trasposizione)	95
6.5.3	Funzioni di coalescenza: gestione dei valori <code>NULL</code>	96
6.6	Funzioni Analitiche (Window Functions)	97
6.6.1	L'Ordinamento interno alla Finestra (<code>ORDER BY</code>)	98
6.6.2	Il Framing (<code>ROWS BETWEEN / RANGE BETWEEN</code>)	99
6.6.3	Funzioni di Ranking (<code>ROW_NUMBER()</code> , <code>RANK()</code> , <code>DENSE_RANK()</code>)	101
6.6.4	Funzioni Posizionali (<code>LEAD()</code> e <code>LAG()</code>)	103
A	Bozze	105
A.1	L'Architettura Client-Server	105
A.1.1	Il Trade-off Architetture: Prestazioni vs Scalabilità	105
A.1.2	Il Confine tra Backend e Database	106
A.2	Logica Attiva: I Trigger	107
A.2.1	Anatomia di un Trigger	108
A.2.2	Esempio Architetture: L'Audit Trail (PostgreSQL)	109
B	Appendice	110
B.1	La Giunzione Naturale in SQL: <code>NATURAL JOIN</code> e <code>USING</code>	110
C	Ottimizzazione, Prestazioni, Scalabilità	112
C.1	L'Anatomia dello Storage	112
C.2	Strutture Dati per l'Indicizzazione	113
C.2.1	Il motore universale relazionale: Il B-Tree (Balanced Tree)	113
C.2.2	Velocità puntuale: Gli Indici Hash	115
C.3	Strategie di Indicizzazione nel Mondo Reale	116
C.3.1	Clustered vs Non-Clustered Index (Primary vs Secondary)	116
C.3.2	Indici Composti e la Regola del Prefisso Sinistro	117
C.4	Ispezione Fisica delle Query: I Piani di Esecuzione (PostgreSQL)	118
C.4.1	Gli strumenti di diagnostica: <code>EXPLAIN</code> vs <code>EXPLAIN ANALYZE</code>	118
C.4.2	Leggere un Piano di Esecuzione e identificare i colli di bottiglia	119

Bozze

A.1 L'Architettura Client-Server

Finora abbiamo analizzato l'algebra relazionale e il linguaggio SQL puro. L'SQL è un linguaggio **dichiarativo**: il programmatore descrive *quale* dato vuole ottenere, lasciando al motore (DBMS) il compito di decidere *come* estrarlo. Proprio per questa sua natura, l'SQL standard è privo dei costrutti di controllo tipici della programmazione imperativa, come i cicli iterativi (**FOR**, **WHILE**) o le ramificazioni condizionali complesse (**IF...ELSE**).

Quando un'applicazione richiede una logica di business complessa (es. "Se il cliente ha un saldo negativo, blocca l'ordine, altrimenti scala l'importo e invia una notifica"), è necessario introdurre un linguaggio di programmazione vero e proprio. In un'architettura software moderna, questa logica può essere ospitata in due luoghi distinti: sul **Lato Server** (all'interno del DBMS) o sul **Lato Client** (all'interno dell'applicativo che si interfaccia al database).

Approccio Lato Server: Linguaggi Procedurali del DBMS In questo scenario, la logica di controllo viene scritta, compilata ed eseguita direttamente all'interno della memoria del database server. I produttori di DBMS hanno esteso lo standard SQL creando dei linguaggi proprietari definiti "Turing completi", che uniscono le direttive di interrogazione ai costrutti della programmazione procedurale.

Esempi principali:

- **PL/SQL** (*Procedural Language/SQL*): Il linguaggio proprietario e altamente ottimizzato sviluppato da Oracle.
- **PL/pgSQL**: L'equivalente implementato nel DBMS open-source PostgreSQL.
- **T-SQL** (*Transact-SQL*): Il dialetto procedurale utilizzato in Microsoft SQL Server.

L'approccio lato server garantisce prestazioni eccezionali nelle operazioni massive sui dati, poiché il codice viene eseguito esattamente dove risiedono le informazioni, eliminando del tutto il traffico di rete tra applicazione e database.

Approccio Lato Client: API e Driver (Il modello Java/JDBC) In questo scenario, il database viene degradato a mero "contenitore stupido" di dati. L'intera logica di controllo (**IF...ELSE**, cicli) viene scritta nel linguaggio generale dell'applicativo (es. Java, Python, C#).

L'applicazione (che funge da *Client* rispetto al database) comunica con il DBMS attraverso specifiche **API** (Application Programming Interface). Nel mondo Java, lo standard assoluto per questa comunicazione è il protocollo **JDBC** (*Java Database Connectivity*). Tramite JDBC, il codice Java apre una connessione di rete, invia una stringa contenente un'istruzione SQL pura, attende che il DBMS la esegua e raccoglie il *Result Set* per elaborarlo.

A.1.1 Il Trade-off Architetture: Prestazioni vs Scalabilità

La scelta tra l'implementazione della logica di business nel database (Lato Server) o nell'applicativo (Lato Client) rappresenta uno dei bivi decisionali più critici nell'ingegneria del software. Le forze in gioco si bilanciano su tre assi fondamentali:

1. Prestazioni Pure (Vantaggio del Lato Server) Se la logica risiede nel DBMS, le elaborazioni avvengono esattamente dove sono memorizzati fisicamente i dati. Questo azzerà il traffico di rete. Calcolare e aggiornare una statistica complessa su un milione di record tramite una procedura PL/SQL interna al database è un'operazione estremamente rapida. Eseguire la stessa operazione dal lato client richiederebbe il trasferimento di un milione di righe attraverso la rete (creando un collo di bottiglia fatale), la loro elaborazione nella RAM dell'applicativo Java e il successivo invio massivo di istruzioni di **UPDATE**.

2. Portabilità e Vendor Lock-in (Vantaggio del Lato Client) Il codice procedurale (es. PL/SQL, T-SQL) è strettamente proprietario. Scrivere la logica di business all'interno del database incatena l'azienda a quello specifico fornitore (fenomeno del **Vendor Lock-in**). Centralizzare la logica in Java, utilizzando librerie standard come JDBC, rende l'applicativo agnostico: è possibile migrare da Oracle a PostgreSQL sostituendo semplicemente il driver, senza dover riscrivere anni di logica aziendale.

3. L'ago della bilancia: La Scalabilità Orizzontale Questo è il fattore ingegneristico che spinge l'industria moderna a preferire quasi sempre l'approccio Lato Client. Se il traffico di un'applicazione esplode, scalare un'architettura backend scritta in Java (es. tramite framework come Spring Boot) è relativamente semplice: è sufficiente istanziare decine di nuovi server applicativi in parallelo, poiché essi sono **stateless** (non mantengono uno stato fisico persistente). Al contrario, il database relazionale è un monolite **stateful** (conserva fisicamente il dato). Scalare orizzontalmente un RDBMS è un'operazione complessa, rischiosa e costosissima. Se si sovraccarica il database costringendolo a fare sia da magazzino che da motore decisionale, esso diventerà inevitabilmente il collo di bottiglia insormontabile dell'intero ecosistema.

✓ La Best Practice dell'Ingegneria Backend

Nei moderni sistemi *Enterprise*, la regola d'oro architetturale impone una rigida separazione delle responsabilità:

- **Il Database funge da "Cassaforte"**: Deve essere lasciato il più "stupido" possibile riguardo alle regole di business, ma deve essere blindato per quanto riguarda la sicurezza e la coerenza del dato (demandata esclusivamente ai vincoli di integrità: chiavi primarie, esterne, vincoli **UNIQUE** e **NOT NULL**).
- **L'Applicativo (Backend) funge da "Cervello"**: Tutte le decisioni operative, i flussi condizionali (**IF...ELSE**), i cicli iterativi e la logica di business pura risiedono nel codice (es. Java), in modo da poter essere versionate, testate automaticamente e, soprattutto, scalate orizzontalmente senza limiti.

A.1.2 Il Confine tra Backend e Database

Nello sviluppo di architetture *Enterprise* (es. stack Java/Spring Boot + PostgreSQL) è un errore comune fare confusione sul concetto di "logica" da delegare al database. Per garantire scalabilità, prestazioni e manutenibilità, l'ingegneria del software impone di tracciare una linea netta tra **Logica di Business** e **Logica dei Dati**.

1. Il Dominio del Backend (Java): Logica di Business e Stato Il server applicativo (il client del database) è l'unico responsabile delle regole aziendali (Business Rules), dei

workflow operativi e delle integrazioni esterne. Esempi tipici includono l'invio di email, l'autorizzazione di un pagamento, o il blocco della cancellazione di un'anagrafica se esistono pratiche pendenti.

Perché delegare a Java?

- **Scalabilità Orizzontale:** In caso di picchi di traffico, è possibile accendere istantaneamente decine di nuove istanze del server Spring Boot per distribuire il carico. Scalare un database relazionale in scrittura, al contrario, è un'operazione estremamente rigida e costosa.
- **Manutenibilità e Testing:** La logica scritta in Java è facilmente testabile tramite *Unit Test* (es. JUnit). Eseguire il debug di logica di business nascosta nel database (tramite *Trigger* o *Stored Procedure*) è un anti-pattern operativo che rallenta lo sviluppo.
- **Prevenzione del Vendor Lock-in:** Scrivere le regole aziendali nevralgiche nel dialetto procedurale di un DBMS (es. PL/pgSQL) lega indissolubilmente l'azienda a quel software, rendendo proibitiva un'eventuale migrazione futura verso un altro motore.

2. Il Dominio del Database (PostgreSQL): Logica dei Dati (Data Crushing) Il database relazionale non deve conoscere le regole di business, ma deve essere sfruttato per la sua capacità nativa di filtrare, aggregare (Aggregations), ruotare (Pivoting) e formattare in modo condizionale (CASE WHEN) moli enormi di dati.

Perché delegare a PostgreSQL?

- **Abbattimento della Latenza di Rete (Network Latency):** Spostare decine di migliaia di righe dal database alla RAM di Java per contarle o raggrupparle satura inutilmente la larghezza di banda. Eseguendo le aggregazioni direttamente in SQL, sul cavo di rete viaggerà solo il risultato finale.
- **Prevenzione della Saturazione della RAM (OutOfMemoryError):** Evitare il *Data Fetching* massivo (ovvero scaricare tabelle intere per elaborarle in memoria tramite *Stream API* e *for-loop*) previene crash disastrosi del server applicativo.
- **Efficienza Computazionale:** I database relazionali (scritti in C/C++) sono dotati di un *Query Optimizer* che analizza gli indici fisici sul disco. Sono ingegnerizzati appositamente per applicare l'algebra relazionale a una velocità inarrivabile per il codice applicativo standard.

Osservazione A.1 (La Regola Aurea dell'Architettura Dati). Il database relazionale non deve mai generare effetti collaterali (Trigger) o governare i flussi decisionali, ma **deve sempre restituire allo strato Java un set di dati strettamente necessario, già filtrato, pre-calcolato e formattato**. In un'architettura moderna, Java rappresenta il cervello del sistema, mentre il DBMS ne è il muscolo matematico.

A.2 Logica Attiva: I Trigger

I vincoli di integrità standard (CHECK, FOREIGN KEY) sono regole **passive**: si limitano a ispezionare il dato e a bloccare l'operazione in caso di violazione. I Trigger, al contrario, introducono il concetto di **Database Attivo**.

Un Trigger è un blocco di codice procedurale che risiede nel database e viene eseguito automaticamente (in gergo, "scatta" o *fires*) in risposta a una specifica operazione di modifica (DML) su una tabella. Il funzionamento dei Trigger si basa sul **Paradigma E-C-A**:

- **E (Evento):** L'azione DML che "risveglia" il trigger (INSERT, UPDATE, DELETE).
- **C (Condizione):** Un test logico opzionale. Se falso, il trigger si ferma senza eseguire l'azione.
- **A (Azione):** Il corpo del codice procedurale da eseguire.

A.2.1 Anatomia di un Trigger

La definizione logica di un trigger richiede la configurazione di quattro parametri strutturali.

1. Il Quando (Timing dell'esecuzione) Definisce l'istante esatto in cui l'azione deve essere intrapresa rispetto all'evento scatenante:

- **BEFORE** (Azione preventiva): Il trigger scatta *prima* che il record venga effettivamente scritto su disco. È lo strumento d'elezione per validare i dati, manipolarli in volo (es. forzare una stringa in maiuscolo) o bloccare preventivamente l'operazione sollevando un'eccezione.
- **AFTER** (Azione consuntiva): Il trigger scatta *dopo* che il database ha salvato il dato e verificato i vincoli standard. Si usa per le propagazioni a catena, come l'aggiornamento di tabelle statistiche, la scrittura di log di sicurezza o la sincronizzazione di giacenze di magazzino.
- **INSTEAD OF** (Azione sostitutiva): Esclusivo per le Viste (VIEW). Intercetta l'operazione DML e, invece di eseguirla, esegue un codice alternativo.

2. L'Evento Scatenante Gli eventi intercettabili sono INSERT ON tabella, DELETE ON tabella e UPDATE ON tabella.

Osservazione A.2 (L'ottimizzazione dell'UPDATE). A differenza di inserimenti e cancellazioni che coinvolgono invariabilmente l'intera tupla, l'operazione di aggiornamento permette una sintassi più granulare: UPDATE OF nome_attributo ON tabella. Questa distinzione è vitale per le prestazioni: permette di "svegliare" il trigger **solo** se viene modificata una colonna specifica d'interesse, ignorando gli aggiornamenti su campi ininfluenti e risparmiando preziosi cicli di CPU.

3. La Granularità (FOR EACH ROW vs Statement Level) Determina quante volte l'azione deve essere eseguita a fronte di un evento che coinvolge molteplici tuple (es. UPDATE esami SET voto = 18 WHERE voto < 18).

- **Statement Level (Di Default):** L'azione viene eseguita **una sola volta** per l'intera istruzione DML, indipendentemente dal fatto che vengano modificate zero, una o un milione di righe.
- **FOR EACH ROW (A livello di Riga):** Il trigger viene messo in un ciclo iterativo iterando l'esecuzione **per ogni singola riga** modificata dall'istruzione.

4. Le Pseudo-Variabili di Stato (OLD e NEW) Se si sceglie la granularità FOR EACH ROW, il DBMS inietta automaticamente nel corpo del trigger due variabili speciali che contengono lo stato della riga:

- **NEW:** Contiene i dati "nuovi" in arrivo (disponibile in INSERT e UPDATE).
- **OLD:** Contiene i dati "vecchi" già presenti su disco (disponibile in UPDATE e DELETE).

A.2.2 Esempio Architetture: L'Audit Trail (PostgreSQL)

! Differenze di Dialecto: Oracle vs PostgreSQL

La sintassi unificata (`CREATE TRIGGER ... BEGIN ... END`) è tipica di Oracle PL/SQL. Nei moderni database open-source come PostgreSQL, l'architettura è più modulare e ingegnerizzata: si crea prima la logica procedurale dentro una **Funzione**, e successivamente si crea il **Trigger** che si limiterà ad "ascoltare" l'evento e invocare la funzione.

Nello sviluppo *Enterprise*, una delle applicazioni più frequenti dei trigger è l'*Audit Trail*: tracciare storicamente le modifiche a campi sensibili (come un salario o il voto di un esame). Vogliamo registrare in una tabella `log_voti` ogni volta che un professore modifica il voto di un esame già registrato.

```
1  -- 1. La Funzione Procedurale (Azione)
2  CREATE FUNCTION traccia_cambio_voto() RETURNS TRIGGER AS $$
3  BEGIN
4      -- Eseguiamo l'azione solo se il voto e' EFFETTIVAMENTE
       cambiato
5      IF NEW.voto <> OLD.voto THEN
6          INSERT INTO log_voti (matricola, corso, vecchio_voto,
              nuovo_voto, data_modifica)
7          VALUES (OLD.matr, OLD.corso, OLD.voto, NEW.voto,
              CURRENT_TIMESTAMP);
8      END IF;
9
10     RETURN NEW; -- Necessario nei trigger a livello di riga
11 END;
12 $$ LANGUAGE plpgsql;
13
14 -- 2. La Definizione del Trigger (Evento e Condizione)
15 CREATE TRIGGER audit_voti_trigger
16 AFTER UPDATE OF voto ON esame -- Evento ottimizzato: scatta solo
       se si tocca 'voto'
17 FOR EACH ROW                  -- Granularita': deve valutare ogni
       riga modificata
18 EXECUTE FUNCTION traccia_cambio_voto();
```

Appendice

B.1 La Giunzione Naturale in SQL: NATURAL JOIN e USING

A differenza dell'algebra relazionale dove l'omissione della condizione trasforma implicitamente un Theta-Join in un Natural Join ($\bowtie$), il linguaggio SQL richiede l'uso di parole chiave esplicite per attivare questo comportamento ed evitare prodotti cartesiani accidentali.

Esistono due approcci sintattici per implementare la fusione delle colonne omonime tipica della giunzione naturale:

L'operatore puro: NATURAL JOIN Questa è la traduzione letterale dell'operatore matematico $\bowtie$.

```
1 SELECT *
2 FROM tabella_1
3 NATURAL JOIN tabella_2;
```

Quando il motore SQL incontra **NATURAL JOIN**, esegue autonomamente due operazioni:

1. **Ricerca degli attributi:** Ispeziona gli schemi delle due relazioni alla ricerca di tutte le colonne aventi **esattamente lo stesso nome**.
2. **Condizione e Fusione:** Genera un'uguaglianza implicita su tali colonne (es. `t1.id = t2.id`) e, nel *Result Set* finale, **fonde** le colonne omonime in una sola, evitando ridondanze.

⚠ La fragilità strutturale del NATURAL JOIN

Nonostante la sua eleganza matematica, l'uso di **NATURAL JOIN** in produzione è universalmente considerato un **anti-pattern ingegneristico**. Il motivo risiede nella sua dipendenza occulta e non controllata dai nomi delle colonne. Se mesi dopo la stesura del codice un amministratore aggiungesse a entrambe le tabelle una colonna di servizio generica (es. `note` o `data_inserimento`), il DBMS modificherebbe silenziosamente la condizione di giunzione pretendendo l'uguaglianza anche su queste nuove colonne (`AND t1.note = t2.note`). Questo distruggerebbe la logica dell'interrogazione originando risultati errati o insiemi vuoti, senza lanciare alcun errore di compilazione.

L'alternativa ingegneristica: La clausola USING Per ovviare alla pericolosità del **NATURAL JOIN** preservandone però il vantaggio sintattico (la fusione delle colonne omonime), lo standard SQL ha introdotto la clausola **USING**.

Questa clausola va a sostituire **ON** e permette al progettista di dichiarare esplicitamente su quali colonne omonime deve avvenire la giunzione, rendendo il codice immune a future modifiche dello schema.

Sintassi con USING:

```
1 -- Supponendo che entrambe le tabelle abbiano un attributo '
   matricola'
2 SELECT *
```

```
3 FROM studente
4 JOIN esame USING (matricola);
```

A differenza della classica clausola `ON` (che restituirebbe due colonne distinte: `studente.matricola` ed `esame.matricola`), la clausola `USING` istruisce il database a **fondere** l'attributo in una singola colonna, rispecchiando in totale sicurezza il comportamento algebrico della giunzione naturale.

Cenni su Ottimizzazione, Prestazioni e Scalabilità dello Storage

Nei capitoli precedenti abbiamo trattato il database relazionale come un'entità logica in grado di restituire i dati interrogati. Tuttavia, nell'ingegneria del software su larga scala, questa astrazione non è più sufficiente. Quando le tabelle passano da poche migliaia a decine di milioni di righe, un'interrogazione scritta in modo superficiale può causare la saturazione delle risorse e il blocco dell'intero sistema.

Questo capitolo si discosta dalla sintassi SQL per esplorare l'architettura fisica dello storage. Comprendere come il Database Management System (DBMS) organizza, indicizza e recupera i byte sul disco rigido o sull'SSD è il requisito fondamentale per poter progettare servizi backend realmente scalabili e per affrontare con successo i colloqui tecnici di livello avanzato.

C.1 L'Anatomia dello Storage

Per comprendere la necessità delle complesse strutture dati utilizzate dai database relazionali moderni, è didatticamente utile partire dall'analisi della forma più elementare possibile di memorizzazione dei dati.

Il Database Primitivo: Il Log Append-Only Immaginiamo di voler costruire il nostro DBMS personale da zero. La struttura di memorizzazione più semplice in assoluto è un normale file di testo in cui ogni riga rappresenta un record (es. un file testuale separato da virgole).

Quando un'applicazione backend invia un comando di `INSERT`, il nostro database rudimentale si limita ad aprire il file e ad accodare la nuova stringa di testo alla fine (operazione di *append*).

Osservazione C.1 (Efficienza estrema in Scrittura). L'accodamento sequenziale (*Append-Only Log*) è in assoluto l'operazione di scrittura più veloce che un supporto di memoria fisico possa eseguire. Non richiedendo alcuna riorganizzazione dei dati preesistenti né complessi posizionamenti della testina del disco, il costo computazionale di un inserimento è minimo e rigorosamente costante nel tempo, denotato in notazione asintotica come $O(1)$.

Il collasso architetturale di questo modello primitivo si manifesta inesorabilmente in fase di lettura. Quando l'applicazione invia una query di estrazione (es. `SELECT * FROM log WHERE id = 12345`), il database primitivo non ha alcuna informazione su dove si trovi fisicamente quel record.

L'unica strategia algoritmica possibile per il motore è aprire il file dalla prima riga e scandirlo integralmente verso il basso finché non trova una corrispondenza. Questa operazione prende il nome di **Ricerca Lineare** (*Sequential Scan*). Se il file è cresciuto fino a contenere 10 milioni di righe, il database potrebbe dover leggere 10 milioni di stringhe per trovare l'unica tupla richiesta. L'algoritmo ha un costo computazionale proporzionale alla mole dei dati, ovvero $O(N)$. All'aumentare del database, il tempo di risposta della singola query degrada in modo lineare, portando in breve tempo il backend al collasso prestazionale (il cosiddetto *Timeout*).

Il Compromesso Fondamentale del Software Engineering Per evitare il disastroso costo della ricerca lineare $O(N)$, l'ingegneria dei dati adotta la stessa identica soluzione utilizzata nell'editoria stampata per trovare un concetto in un libro di mille pagine: si crea un **Indice**.

Un indice, all'interno di un database, è una struttura dati aggiuntiva e fisicamente separata dai dati primari. Il suo unico scopo è mantenere una mappa ordinata (una "segnaletica") che indichi al motore di ricerca l'esatto indirizzo di memoria (*offset*) di ciascun record. Invece di scansionare i dati, il DBMS interroga l'indice, trova le coordinate del dato, e lo recupera a colpo sicuro.

Sebbene l'idea appaia come la soluzione definitiva, la sua implementazione introduce nel sistema quello che Martin Kleppmann definisce il "compromesso fondamentale" (*Trade-off*) dei database. Mantenere una struttura dati aggiuntiva ha un costo architetturale severo. Sebbene un indice abbatta drasticamente le tempistiche di lettura, esso **degrada le prestazioni in scrittura**.

Ogni volta che l'applicazione esegue un'istruzione INSERT, UPDATE o DELETE, il DBMS non può più limitarsi ad accodare velocemente il dato nel file primario ($O(1)$), ma è fisicamente costretto a ri-calcolare, ri-bilanciare e aggiornare simultaneamente tutti gli indici associati a quella tabella affinché la mappa rimanga coerente.

Non esiste un database relazionale "ottimizzato per impostazione predefinita". La creazione degli indici è un'operazione manuale e rappresenta una precisa responsabilità dell'ingegnere software o del Database Administrator (DBA), il quale deve bilanciare l'infrastruttura basandosi sui reali *pattern* di interrogazione dell'applicazione.

La regola aurea della scalabilità afferma che: *"Un indice ben scelto velocizza enormemente le query di lettura, ma ogni singolo indice aggiunto rallenta proporzionalmente le operazioni di scrittura"*. Creare indici a tappeto su ogni colonna "per sicurezza" è un grave anti-pattern che rischia di paralizzare l'inserimento dei dati nel sistema.

C.2 Strutture Dati per l'Indicizzazione

Per evitare la scansione lineare, il DBMS deve organizzare gli indici tramite strutture dati avanzate. Sebbene esistano diversi approcci (come i log LSM utilizzati nei DB NoSQL), il mondo relazionale è dominato da decenni da un unico, indiscusso standard algoritmico.

C.2.1 Il motore universale relazionale: Il B-Tree (Balanced Tree)

Introdotta nei primi anni '70 e costantemente raffinata, il **B-Tree** (precisamente nella sua variante B^+ -Tree) è l'albero di ricerca bilanciato su cui poggia l'intera architettura di PostgreSQL.

A differenza degli alberi binari di ricerca studiati nei corsi di informatica teorica, il B-Tree è progettato specificamente per lavorare in simbiosi con l'hardware dei supporti di memoria fisici (Dischi Meccanici ed SSD).

1. L'organizzazione fisica sul disco: Le Pagine Il segreto del B-Tree risiede nella sua granularità. Mentre i log di testo crescono in modo indefinito, il B-Tree scompone l'intero database in blocchi di memoria a dimensione fissa, chiamati **Pagine** (o *Blocks*), tipicamente di 4 KB o 8 KB (in PostgreSQL il default è 8 KB). Il DBMS legge o scrive sul disco sempre una pagina intera alla volta.

La struttura si sviluppa in modo gerarchico e piramidale:

- **Nodo Radice (Root Node):** È la pagina iniziale dell'indice. Contiene una sequenza ordinata di chiavi e una serie di puntatori (indirizzi di memoria del disco) che rimandano alle pagine del livello inferiore.
- **Nodi Interni (Internal Nodes):** Pagine intermedie che fungono da "svincoli autostradali". Leggendo la chiave cercata, indirizzano il motore verso la pagina successiva, restringendo progressivamente il campo di ricerca.
- **Nodi Foglia (Leaf Nodes):** Sono le pagine situate alla base della piramide. Contengono le chiavi effettive e, accanto a ciascuna di esse, il puntatore fisico (*Tuple ID* o *offset*) che indica dove si trova la riga reale all'interno della tabella principale. Nelle varianti moderne, tutte le foglie sono collegate tra loro da puntatori orizzontali per velocizzare la scansione sequenziale ordinata.

Il numero di puntatori a figli che una singola pagina può ospitare prende il nome di **Branching Factor** (o *Fan-out*). Poiché una pagina di 8 KB può contenere centinaia di chiavi numeriche, il fattore di ramificazione è straordinariamente elevato (spesso superiore a 200–300).

Esempio C.1 (Tracciamento di una ricerca puntuale nel B-Tree). Supponiamo che il backend Java esegua l'interrogazione `SELECT * FROM sinistro WHERE id_cliente = 83`. La colonna `id_cliente` è indicizzata tramite un B-Tree. Vediamo come il motore di PostgreSQL naviga fisicamente le pagine da 8 KB memorizzate sul disco per recuperare il record.

Fase 1: Ispezione del Nodo Radice (Pagina #100) Il motore carica in RAM la pagina di radice. Al suo interno trova una guida sintetica suddivisa in intervalli di chiavi:

- Chiavi < 50 → Punta alla Pagina #101
- Chiavi da 50 a 99 → Punta alla Pagina #102
- Chiavi ≥ 100 → Punta alla Pagina #103

Poiché il valore cercato è 83, il Query Planner scarta immediatamente le pagine #101 e #103 (e tutti i sotto-alberi ad esse collegati). Il motore esegue una lettura sul disco per caricare solo la **Pagina #102**.

Fase 2: Navigazione del Nodo Interno (Pagina #102) La Pagina #102 è un nodo intermedio (lo svincolo). Al suo interno la granularità aumenta e l'intervallo viene ulteriormente frammentato:

- Chiavi da 50 a 69 → Punta alla Pagina #201
- Chiavi da 70 a 89 → Punta alla Pagina #202
- Chiavi da 90 a 99 → Punta alla Pagina #203

Il valore 83 ricade nell'intervallo centrale. Il motore ignora le pagine #201 e #203 ed effettua un secondo accesso al disco per caricare la **Pagina #202**.

Fase 3: Identificazione nel Nodo Foglia (Pagina #202) La Pagina #202 è un nodo foglia, la base della piramide. Qui non ci sono più puntatori ad altre pagine dell'indice, ma le chiavi reali associate agli indirizzi fisici dei dati nella tabella (*Heap File*):

- Chiave 75 → Record memorizzato all'offset 0x0A2
- Chiave 83 → Record memorizzato all'offset 0x1F4
- Chiave 89 → Record memorizzato all'offset 0x33B

Il motore trova la corrispondenza esatta per la chiave 83 e ottiene il puntatore fisico 0x1F4.

Fase 4: Estrazione dal File dei Dati (Heap File) L'indice ha terminato il suo compito. Il database esegue l'ultimo accesso al disco puntando direttamente all'offset 0x1F4 della tabella `sinistro`, estrae la riga completa (targa, data, danno stimato) e la impacchetta nel *Result Set* da inviare sulla rete all'applicazione Java.

Grazie a un elevato *Branching Factor*, l'albero si sviluppa pochissimo in altezza (profondità) ma immensamente in larghezza. Un B-Tree a soli 4 livelli logici, con un fattore di ramificazione pari a 250, è matematicamente in grado di ospitare oltre 3.9 miliardi di record.

Cercare un ID specifico in un database da 4 miliardi di righe non richiede 4 miliardi di scansioni ($O(N)$), ma l'apertura di **sole 4 pagine sul disco**. Il costo computazionale è rigidamente logaritmico, denotato come $O(\log N)$. Le prestazioni rimangono stabili, fulminee e indipendenti dalla crescita del database.

2. Il comportamento in Scrittura: Il Page Split Il B-Tree viene definito "Bilanciato" perché l'algoritmo garantisce che tutte le pagine foglia si trovino sempre, in ogni istante di tempo, alla stessa identica distanza (profondità) dalla radice. Questo previene il degrado dell'albero.

Tuttavia, mantenere questo perfetto bilanciamento durante un'operazione di **INSERT** richiede un calcolo computazionale oneroso. Quando l'applicazione backend inserisce una nuova riga, il DBMS naviga il B-Tree per individuare la pagina foglia corretta in cui allocare la nuova chiave in ordine numerico.

A questo punto si aprono due scenari fisici:

1. **Spazio disponibile:** Se la pagina da 8 KB ha ancora byte liberi, la chiave viene semplicemente scritta nella pagina. Il costo è minimo.
2. **Pagina satura (Il Page Split):** Se la pagina è fisicamente piena, il motore non può semplicemente accodare il dato. Deve attivare l'algoritmo di **Page Split** (Scissione della Pagina): la pagina viene allocata *ex novo* sul disco, i dati vengono divisi a metà tra la vecchia e la nuova pagina, e la chiave mediana viene spinta verso l'alto nel nodo padre per aggiornare i puntatori.



Il collasso da Page Split a cascata

Se anche il nodo padre è saturo, il fenomeno del *Page Split* si propaga ricorsivamente verso l'alto, potendo arrivare a sdoppiare il nodo radice stesso e aumentando l'altezza dell'intero albero.

Durante un inserimento che causa un Page Split a cascata, il database deve effettuare molteplici operazioni di scrittura in punti diversi del disco rigido anziché un singolo accodamento rapido. Questo spiega sperimentalmente perché le **INSERT** massive rallentino drasticamente in presenza di troppi indici attivi.

C.2.2 Velocità puntuale: Gli Indici Hash

Mentre il B-Tree domina il mondo relazionale per la sua straordinaria versatilità, esiste un'architettura di indicizzazione alternativa progettata per massimizzare la velocità assoluta in scenari estremamente specifici: l'**Indice Hash**.

Basato sulla celebre struttura dati della *Hash Table* (Tabella Hash), questo indice abbandona completamente l'idea di navigare un albero strutturato pagina per pagina. Al suo posto, utilizza una **Funzione di Hash**: un algoritmo che prende in input la chiave di ricerca (es. la *matricola* di uno studente) e la trasforma istantaneamente in un indirizzo di memoria (un *offset* o un *bucket*) che punta direttamente alla posizione del record sul disco o nella RAM.

Poiché l'indirizzo viene calcolato matematicamente e non cercato fisicamente tramite salti tra le pagine, l'indice Hash evita la fase di navigazione a strati del B-Tree. Il costo computazionale per recuperare un record tramite un indice Hash è **Tempo Costante** ($O(1)$). Che il database contenga dieci righe o dieci miliardi di righe, il tempo medio per decodificare la chiave e trovare il record esatto rimane identico e pressoché istantaneo.

Se le prestazioni per le ricerche puntuali (*Point Queries*, es. `SELECT * FROM studente WHERE matricola = 10001`) sono matematicamente inarrivabili per qualsiasi altra struttura, per quale motivo i database relazionali come PostgreSQL utilizzano il B-Tree come impostazione di default?

La risposta risiede nel limite strutturale e insuperabile delle funzioni di Hash: **la distruzione dell'ordine naturale dei dati**.

Per definizione crittografica e matematica, una buona funzione di Hash distribuisce i valori in modo apparentemente casuale e uniforme per evitare collisioni di memoria. Se abbiamo due matricole consecutive come 10001 e 10002, i loro hash genereranno locazioni di memoria completamente diverse e fisicamente distanti tra loro.

Questo significa che l'indice Hash non ha la benché minima idea concettuale di cosa significhi "maggiore di", "minore di" o "vicino a". Se un'applicazione backend esegue una *Range Query* (un'interrogazione a intervalli, es. `SELECT * FROM studente WHERE dataN BETWEEN '1999-01-01' AND '2000-12-31'`), l'indice Hash risulta **completamente inutile**. Il motore del database, incapace di sfruttare l'indice per definire un perimetro logico, sarà costretto a declassare l'operazione a una letale ricerca lineare su tutto il disco (*Sequential Scan*).

Nell'ingegneria del backend moderno, definire esplicitamente degli indici Hash all'interno di un database relazionale (es. `CREATE INDEX idx_nome ON tabella USING HASH (colonna);`) è una pratica rara. Vengono presi in considerazione solo se l'architetto del software ha la certezza matematica che quella specifica colonna verrà interrogata **esclusivamente tramite operatori di uguaglianza** (=) e mai tramite operatori di disuguaglianza (>, <, BETWEEN) o direttive di ordinamento (ORDER BY).

Il paradigma Hash trova invece la sua massima e naturale espressione nei database NoSQL in-memory di tipo *Key-Value Store* (come **Redis** o **Memcached**), dove le query ad intervalli non sono strutturalmente previste dal motore di archiviazione e l'obiettivo primario è la velocità di lettura assoluta in cache.

C.3 Strategie di Indicizzazione nel Mondo Reale

Conoscere il funzionamento logaritmico $O(\log N)$ di un B-Tree è solo il primo passo. L'ingegneria del database richiede di comprendere come l'albero si leghi fisicamente ai dati reali scritti sul disco. La distinzione fondamentale riguarda il contenuto delle "foglie" dell'albero: contengono i dati veri e propri o solo delle coordinate?

C.3.1 Clustered vs Non-Clustered Index (Primary vs Secondary)

Quando l'applicazione backend inserisce una nuova riga, il motore relazionale deve decidere dove scriverla fisicamente sull'hard disk. Esistono due architetture principali per la gestione di questo salvataggio:

1. Il File di Heap (Heap File) e gli Indici Secondari In questa architettura (che è il comportamento **di default in PostgreSQL**), la tabella vera e propria è un *Heap File*: un file non ordinato in cui le nuove righe vengono semplicemente accodate nel primo

blocco di memoria libero disponibile. L'indice (es. sulla Chiave Primaria) viene costruito come una struttura B-Tree separata. Le foglie di questo albero non contengono i dati della riga, ma solo un puntatore (chiamato *Tuple ID* o *TID*) che indica l'esatto offset nel file di Heap. Gli indici che "puntano" a un file separato sono definiti **Indici Secondari** (*Non-Clustered Indexes*).

2. Tabelle Organizzate a Indice (Index-Organized Tables / Clustered) In questa architettura (che è il default, ad esempio, nel motore InnoDB di MySQL), il file di Heap non esiste. La tabella è l'indice stesso. L'albero B-Tree viene costruito sulla Chiave Primaria e le sue foglie contengono nativamente l'intera riga dei dati (es. nome, cognome, indirizzo). Questo si definisce **Indice Clustered** (o Primario). Poiché i dati vivono direttamente nelle foglie dell'albero, la tabella è fisicamente costretta ad essere ordinata sul disco in base alla Chiave Primaria.

Osservazione C.2 (Il costo del Pointer Lookup). Perché questa distinzione è vitale per le prestazioni? Se un'interrogazione utilizza un indice secondario (come fa nativamente PostgreSQL) per trovare un record, il motore deve prima attraversare il B-Tree per trovare il puntatore, e **successivamente** eseguire un ulteriore salto sul disco verso l'Heap File per estrarre le colonne richieste. Questo "salto extra" (*Pointer Lookup*) su dischi meccanici può risultare un collo di bottiglia severo se la query estrae decine di migliaia di righe sparse in blocchi diversi del disco.

C.3.2 Indici Composti e la Regola del Prefisso Sinistro

Nel business reale, le interrogazioni filtrano raramente per un singolo attributo. Spesso le clausole **WHERE** combinano più colonne (es. **WHERE** cognome = 'Rossi' **AND** città = 'Roma'). Per ottimizzare queste query, il DBMS permette di creare **Indici Composti** (*Multi-column indexes*), ovvero un singolo B-Tree che indicizza la concatenazione rigorosa di più attributi.

```
1 -- Creazione di un indice composto su due colonne in PostgreSQL
2 CREATE INDEX idx_cognome_nome ON studente (cognome, nome);
```

Il funzionamento di un indice composto è concettualmente identico all'elenco telefonico cartaceo: i dati sono ordinati **primariamente** per la prima colonna (cognome). A parità di cognome (es. tutti i "Rossi"), i record interni vengono ordinati **secondariamente** in base alla seconda colonna (nome).

⚠ La trappola mortale: La Regola del Prefisso Sinistro

L'ordine in cui si dichiarano le colonne durante la creazione di un indice composto è architetturalmente vincolante. Un indice composto è utilizzabile dal Query Planner **se e solo se** l'interrogazione filtra per la prima colonna dell'indice (o per un prefisso continuo letto rigorosamente da sinistra verso destra).

Data la presenza dell'indice `idx_cognome_nome(cognome, nome)`:

- **WHERE** cognome = 'Rossi': **Veloce**. L'indice viene usato parzialmente. Il motore salta direttamente alla sezione dei "Rossi".
- **WHERE** cognome = 'Rossi' **AND** nome = 'Mario': **Velocissimo**. L'indice viene sfruttato in modo ottimale.
- **WHERE** nome = 'Mario': **Lento (Sequential Scan)**. L'indice è totalmen-



te inutilizzabile. Senza conoscere prima il cognome, i nomi "Mario" sono sparsi casualmente in tutto l'elenco telefonico. Il database sarà costretto a scansionare fisicamente l'intera tabella.



Design degli Indici Composti

Quando si progetta un indice multicolonna per un'applicazione backend, la colonna posizionata all'estrema sinistra (*Leading Column*) dovrebbe sempre essere l'attributo interrogato più di frequente in modo isolato, o quello con la **selettività maggiore** (l'attributo in grado di scartare matematicamente il maggior numero di righe fin dal primo filtraggio). Se la logica di business impone di cercare spesso gli utenti *esclusivamente* per nome, sarà architetturalmente obbligatorio istanziare un secondo indice separato e dedicato alla colonna **nome**.

C.4 Ispezione Fisica delle Query: I Piani di Esecuzione (PostgreSQL)

Comprendere la teoria degli indici è il fondamento dell'ingegneria del software, ma nei sistemi di produzione la realtà fisica spesso diverge dalla teoria. Il motore del database potrebbe decidere, contro ogni nostra previsione, di ignorare un indice perfettamente costruito. Per diagnosticare i colli di bottiglia e validare le nostre scelte architetturali, PostgreSQL ci fornisce lo strumento di introspezione definitivo: il comando **EXPLAIN**.

C.4.1 Gli strumenti di diagnostica: EXPLAIN vs EXPLAIN ANALYZE

Quando inviamo una query, il *Query Planner* (il cervello matematico del database) valuta diversi algoritmi possibili per estrarre i dati e sceglie quello con il costo computazionale stimato minore. Questo piano di battaglia prende il nome di **Piano di Esecuzione** (*Execution Plan*).

- **EXPLAIN:** Antepoendo questa parola chiave a qualsiasi interrogazione, PostgreSQL non eseguirà fisicamente la query, ma restituirà una stringa di testo contenente la *stima* teorica del piano. Mostrerà quali indici intende usare, quante righe stima di dover attraversare e quale costo computazionale teorico ha calcolato.
- **EXPLAIN ANALYZE:** Questa è la direttiva di profilazione reale e inconfutabile. Il motore eseguirà fisicamente la query e restituirà un report comparativo: mostrerà la stima teorica a fianco del tempo di esecuzione effettivo (in millisecondi) e del numero di righe processate sul campo.

```
1 -- Richiesta del piano di esecuzione reale per un'estrazione
2 EXPLAIN ANALYZE
3 SELECT nome, cognome FROM studente WHERE città = 'Roma';
```



Il pericolo nascosto di EXPLAIN ANALYZE

Poiché **EXPLAIN ANALYZE** esegue realmente le operazioni sull'hardware per misurarne le tempistiche, utilizzarlo su istruzioni DML come **UPDATE**, **INSERT** o **DELETE** causerà l'effettiva modifica irreversibile dei dati di produzione. Per ispezionare la reale esecuzione di un'operazione di scrittura distruttiva senza alterare lo stato del



database, è obbligatorio avvolgerla in una transazione isolata (`BEGIN;`) seguita da un immediato annullamento (`ROLLBACK;`).

C.4.2 Leggere un Piano di Esecuzione e identificare i colli di bottiglia

L'output di un piano di esecuzione si legge dal basso verso l'alto (eseguendo un'analisi dai nodi foglia dell'albero logico fino all'estrazione finale). Analizzando il responso, l'ingegnere software deve ricercare specifiche parole chiave per determinare lo stato di salute dell'interrogazione:

1. Sequential Scan (Seq Scan) Indica che il database ha ignorato gli indici a disposizione e sta scansionando l'intero file fisico della tabella dall'inizio alla fine (Ricerca Lineare con complessità $O(N)$). *Diagnosi Oggettiva:* Se la tabella conta poche migliaia di righe, il Query Planner opta deliberatamente per il **Seq Scan** perché risulta meccanicamente più veloce caricare l'intero blocco in RAM piuttosto che sostenere il costo di un *Pointer Lookup*. Se invece questa dicitura compare su tabelle da milioni di record, è in corso un'emergenza prestazionale causata dall'assenza di un indice idoneo o da una violazione della regola del prefisso sinistro negli indici composti.

2. Index Scan Indica che il motore sta attraversando la struttura logaritmica del B-Tree ($O(\log N)$) e sta impiegando i puntatori per saltare nel file di Heap ed estrarre i blocchi di dati reali. *Diagnosi Oggettiva:* È l'esecuzione architetturale ottimale standard. La query risulterà scalabile, con latenze minime garantite all'aumentare dei volumi.

3. Index Only Scan (L'Esecuzione Perfetta) Costituisce l'operazione computazionale più efficiente offerta dal motore relazionale. Si verifica quando il *Query Planner* constata che **tutte** le colonne richieste dalla clausola **SELECT** sono già intrinsecamente memorizzate all'interno dei nodi dell'indice stesso. *Diagnosi Oggettiva:* Il database estrae i byte direttamente dalla struttura del B-Tree e confeziona il risultato ignorando completamente l'esistenza del file della tabella principale (*Heap File*). Questo approccio abbattere drasticamente le operazioni di input/output meccanico (I/O) sul disco.



Il Covering Index

Se i pattern di carico applicativo prevedono milioni di invocazioni quotidiane per una specifica estrazione (es. il recupero di nome e cognome dato un identificativo primario), l'ingegnere può forzare artificialmente un **Index Only Scan** strutturando un **Covering Index** (Indice di Copertura).

Questo si realizza istanziando le colonne di filtro nell'indice principale e aggiungendo le colonne di *payload* come appendice ai nodi. In PostgreSQL si adopera la clausola **INCLUDE**:

```
1 CREATE INDEX idx_matr_copertura ON studente (matricola)
   INCLUDE (nome, cognome);
```

Questa operazione genererà ridondanza fisica sul disco (duplicando i valori di nome e cognome nell'indice), ma ottimizzerà le prestazioni di lettura in modo assoluto e inconfutabile.